H2S-NN Revision: Progress Report

Making the model scalable, letting path information reach the nodes, and fixing the data pipeline

8 November 2026 Measurements: ACM dataset, NVIDIA RTX 4060 Laptop GPU (8 GB)

Overview

The reviewers' main concerns included whether H2S-NN scales to larger graphs. While preparing the revision, we also found a structural problem: in the submitted model, the term that was meant to carry information from the meta-path instances (the 2-cells) to the nodes had exactly zero effect during diffusion. Two main changes address these, followed by three supporting changes (Section 3):

1. **Lighter restriction maps.** Each edge no longer stores its own full `d×d` matrix.
2. **Meta-path faces coupled directly to nodes.** Each meta-path instance is still a face (2-cell) of the model. It now passes information straight to every node on it, through maps that the face itself determines, and the edge-level machinery of the old model is removed.

15.9×	**5.5×**	**4.9×**	**5.3×**
less memory for the restriction maps	less GPU memory for the whole model	faster per training step	fewer parameters (1.62M → 0.31M)

Whole-model memory and speed are compared at stalk dimension d = 8, the largest size at which the old model still fits on our 8 GB GPU. The map memory and parameter counts are for d = 32, the setting used in the paper. At d = 32 the old model does not fit on the GPU at all, while the new model needs 2.5 GB.

1. Lighter restriction maps

Before

For every edge, the model learned two restriction maps, one for each endpoint. Each map was a full `d×d` matrix. On ACM there are 547,872 edges, so at d = 32 the maps alone need about 1.1 billion numbers (4.2 GB). This is the main reason the submitted model could not run on large heterophilic benchmarks such as ogbn-mag.

After

We split each map into three parts, and only the small parts are stored per edge:

```
map for node v on edge e  =  c_e  ×  R_relation  ×  (k reflections built from node features)
```

- **A rotation per relation type** (R): for example, all paper→author edges share one. ACM needs only 16 such matrices in total, instead of two per edge.
- **k reflections per edge**: these make the map depend on the actual nodes. A reflection is a `d×d` matrix, but it is fully described by a single vector of d numbers. It can be

applied to a feature vector directly ($x - 2u(u^T x)/\|u\|^2$), so the matrix is never built. We use k = 2.

- **One scale per edge** (c_e): a single positive number, shared by both endpoints. Sharing it keeps the transport along a path a pure rotation, so long paths cannot blow up numerically.

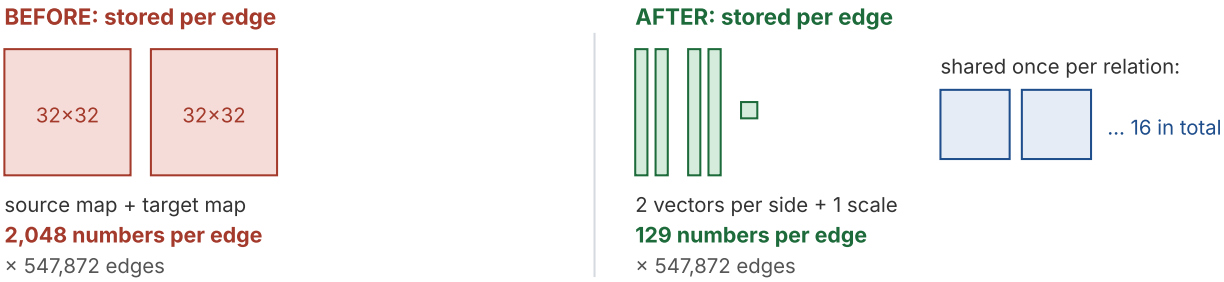


Figure 1. What is stored for each edge at d = 32. The large matrices become shared per relation type; each edge keeps only a few vectors.

Choosing k

k sets the trade-off between flexibility and memory. With k = d reflections, the map could be any rotation, which is as expressive as the old model but saves nothing. With a small k, edges of the same relation share the rotation and differ only in a few directions.

Restriction maps on ACM, d = 32	Numbers stored	Reduction
Before: full matrices	1,122,041,856	—
After, k = 1	35,628,064	31.5×
After, k = 2 (chosen)	**70,691,872**	**15.9×**
After, k = 4	140,819,488	8.0×

Result of this change alone

At d = 8, whole-model GPU memory halved (6.08 → 3.00 GB) and a training step became 1.8× faster (3.18 → 1.73 s). The part of the computation that builds and applies the maps became 5× faster. Total memory did not fall by the full 15.9×, because the rest of the old model (edge states and the dense maps on the path side) still dominated. The second change removes that part.

2. Meta-path faces coupled directly to nodes

What a meta-path face means

Each meta-path instance, for example P_1–A–P_2 (two papers with a common author), is a face (2-cell). Its boundary is the instance itself together with a **closure edge** P_1–P_2, which stands for the meta-path relation "P_1 and P_2 are related through Paper–Author–Paper". The face states a coherence condition: going from P_1 to P_2 along the meta-path relation must agree with going through the instance. In sheaf terms, this condition forces the map of the closure edge to equal the composition of the maps along the instance (the path's *holonomy*). The meta-path relation is therefore not a new relation learned freely, as in HAN, but is fixed by the instance that witnesses it, including its intermediate node and

the order of its relations. This meaning is the same before and after the change; what changes is how the face reaches the nodes.

Before

The face had its own term, but that term influenced the nodes only indirectly: face → states stored on edges → nodes. This route has a mathematical problem. When the diffusion is linear, which it is in our model, the face term's influence on the nodes is **exactly zero**. This holds for any number of diffusion steps, and we also confirmed it numerically. Path information still reached the prediction through two other routes:

- the closure edge P_1–P_2, used as an ordinary edge of the graph operator. Its map was the holonomy, as the face requires, but the face's own term added nothing beyond this edge;
- a single average of the edge states in the final classifier.

In addition, the edge route required storing a large tensor on every edge and every path, which used most of the GPU memory.

After

The face is now coupled directly to **every** node on its boundary path: both endpoints and the intermediate node(s). The coupling uses the face's own restriction maps. These are not new parameters: the face's coherence condition forces them to be the transports along the path, $T_i = Q_1 Q_2 \cdots Q_i$, the same maps that fix the closure edge. Each diffusion step does two things:

1. It forms the face's value z_f: the average of its nodes' features after transporting them into the face, so that relation order and direction matter. z_f lives in the face's own space.
2. It pulls every node on the path toward z_f (transported back to that node), together with the usual pull toward its graph neighbours.

Both pulls together form a single **sheaf Laplacian**. The cellular sheaf on the 2-complex (nodes, edges, closure edges and faces, with their maps) is kept as it was. What is removed is the computation on edges: the edge states, the edge Laplacian and the dense path-side matrices.

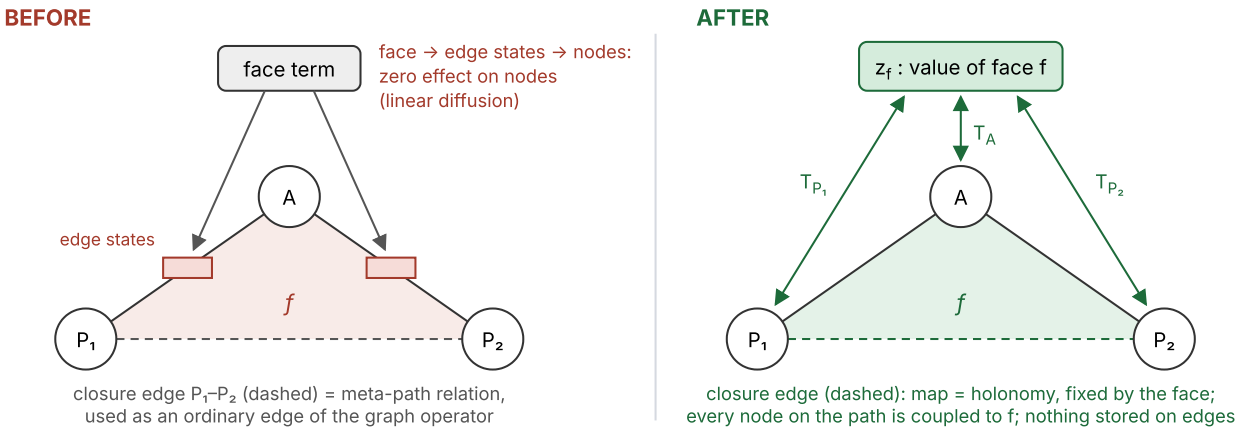


Figure 2. How a meta-path face (Paper–Author–Paper) influences the nodes, before and after. In both, the face f is bounded by the instance P₁–A–P₂ and the closure edge P₁–P₂, whose map the face fixes to the holonomy of the path. After the change, the face is coupled to every node on the path through the maps T, which the face also fixes; information flows both ways.

Properties we proved and checked numerically

- **It respects the sheaf structure.** The new term never contradicts the graph's own sheaf: any assignment that is consistent on the graph stays consistent. If the maps on the path were chosen freely, as in a hypergraph model, this property would be lost. We checked both cases.
- **It is stable.** After normalization, the operator's largest eigenvalue is at most 2, so the diffusion step cannot diverge for any learned step size. The largest value observed over 60 random test graphs was 1.99999. Without normalization, values reached 10^{16}.
- **The face reaches the nodes.** In one step, the face carries information between all nodes on the path. In the old model the face term had zero effect on the nodes.
- **The intermediate node shapes the meta-path relation.** The holonomy from P₁ to P₂ is the composition of the maps of both boundary edges, and these depend on the intermediate node's features. Two Paper–Author–Paper instances with different authors therefore give different relations between their papers. (The intermediate node also reaches the endpoints in one step through the ordinary edges; that part is not new.)

What this change gives up, and what it keeps

The face term can be rewritten exactly as pairwise terms between all nodes on the path (n = number of nodes on the path):

```
Σ_i ‖T_i x_i − z_f‖²  =  (1/n) Σ_{i<j} ‖T_i x_i − T_j x_j‖²
```

For P₁–A–P₂ the three pairs are P₁–A and A–P₂, with exactly the maps of the existing edges, and P₁–P₂, with the holonomy map, that is, the closure edge. So the face reaches the nodes through the meta-path relation it defines and by strengthening the edges of its own instance, each pair with weight `γ·w_f/3`. This is what a meta-path face is meant to do. The difference from HAN-style meta-path edges is therefore not that the endpoint relation is absent; it is that the relation's map is fixed by the instance instead of being learned freely. Several consequences follow, and the paper will state them:

- **Values on edges and faces are no longer evolved.** The sheaf on the 2-complex is unchanged, but the model no longer runs the Hodge Laplacian on edge and face

values. In the submitted model that part never reached the nodes (see Before), so no effect on the nodes is lost. What is lost is the option to read edge or face values out directly.

- **The face is a definition, not a measurement.** The closure edge is not observed in the data; its map is defined by the face, so every face is consistent by construction. A face cannot detect conflicting relations. It imposes that the meta-path relation agrees with its instance.
- **The same operator could be written without faces**, as transports along paths. The face is what justifies this choice: its coherence condition allows only the path transport as the map of the meta-path relation, and a free choice breaks the sheaf property listed above.
- **The meta-path relation may now be weaker.** In the submitted model the closure edge was a full edge of the graph operator; now it carries weight `γ·w_f/n`. Whether this helps or hurts accuracy has not been measured. The ablations in Section 5 compare the two directly.

Result

Stalk dimension d	Old model	After change 1	After both changes
32 (paper setting)	**does not fit in 8 GB**	**does not fit in 8 GB**	**2.5 GB, 1.8 s**
16	does not fit in 8 GB	6.6 GB, 6.0 s	1.4 GB, 1.0 s
8	6.1 GB, 3.2 s	3.0 GB, 1.7 s	1.1 GB, 0.65 s
Parameters at d = 32	1.62M	0.46M	0.31M

Peak GPU memory and time for one training step (forward and backward) on ACM, averaged over 5 runs.

At d = 8, the new model uses 5.5× less memory and is 4.9× faster than the old one. At d = 32, the setting in the paper, the old model no longer fits on our GPU, while the new one needs 2.5 GB. A 20-epoch training run was stable throughout.

3. Further changes: graph encoding, path sampling and node gates

Once faces were coupled directly to nodes, three more parts of the pipeline needed attention: how the graph's edges are represented, how path instances are sampled and computed, and whether influence between nodes has to be symmetric.

3.1 Each fact is stored once

Before

The HGB benchmarks store every relation in both directions. In ACM, for example, the relation *paper* → *author* (PA) has an exact mirror *author* → *paper* (AP) that contains the same pairs reversed. All 10 relation pairs in ACM, DBLP and IMDB are exact mirrors of this kind. Standard message-passing GNNs need both directions, because messages flow from source to target only. The old model inherited this and treated the single fact "paper p is written by author a" as two separate edges, each with its own unrelated maps.

Why it is a problem

The sheaf Laplacian is already symmetric, so the second copy adds no direction, only a second, conflicting constraint. For one fact, the energy has two terms:

```
E(x) = ‖F¹_src x_p − F¹_tgt x_a‖²  +  ‖F²_src x_a − F²_tgt x_p‖²
```

A consistent assignment ($E = 0$) must satisfy $x_a = Q_1 x_p$ and $x_a = Q_2 x_p$ at the same time. With two independently learned maps, $Q_1 \neq Q_2$, and generically only $x = 0$ survives. On a single link at $d = 8$, the space of consistent assignments collapsed from 8 dimensions to 0, which makes the theory's guarantee that "path cells never contradict the graph sheaf" empty.

After

Each fact is now one edge. Two relations are recognised as inverses by one exact, parameter-free rule:

```
ρ(r) = r′   if and only if   the pairs of r′ are exactly the pairs of r reversed
a fact:  (u, r, v) ≡ (v, ρ(r), u)   →   stored once, as one oriented edge
```

Direction is kept: when a path walks along an edge against its stored orientation, it uses the reverse transport $Q^\top$ instead of Q. If two relations are not exact mirrors, they stay separate.

Result

- ACM goes from 547,872 stored edges to 273,936 facts, so the graph operator is half the size.
- **We found a data leak in link prediction.** When a link (p, a) was held out for testing, the old split removed it from PA but left its mirror (a, p) in AP, so the model could see the answer. On a test case, 8 of 21 held-out links leaked. Splits are now made per fact. Earlier link-prediction numbers, ours and the baselines', may be inflated on datasets with mirror relations and will be rerun.
- **We also found a loader bug in MUTAG.** Bond types were assigned by position after PyTorch Geometric had reordered the edges, so 1,771 of 7,442 bonds had the wrong type, in all 188 graphs. All earlier MUTAG results will be rerun.
- Baselines still receive the standard stored graph with both directions, so every method sees the same facts.

3.2 Path instances: unbiased sampling and faster computation

Before

Path instances were collected by depth-first search with a fixed quota of r paths per starting node. This has three problems.

- **It is biased.** Take a paper with 3 authors, each with 30 papers, and $r = 20$. The search fills the quota through the first author and never reaches the other two. In 200 of 200 random seeds, only 1 of the 3 authors was used as the middle node. With one author who has 300 papers, 48–50 of 50 paths went through that author. The question "does

the middle node matter?", which the reviewers raised, cannot be answered if the sampler always picks the same middle node.

- **Duplicates are counted twice.** The path p_1–a–p_2 and its reverse p_2–a–p_1 became two cells (3,690 of 16,300 in a test), so their weight depended on chance.
- **It is slow**: it is a Python loop per starting node, and was the preprocessing bottleneck for larger graphs.

After: sampling

Paths are drawn by random walks that follow the meta-path's relation types and directions. Each step picks uniformly among the allowed neighbours, so every path has a known probability:

```
p(path | start a, meta-path π)  =  ∏ over steps j  of  1 / (number of allowed
neighbours at step j)
r walks per (start node, meta-path), each with weight 1/r
identical paths (in either direction) are merged by ADDING their weights:  m_f = Σ
weights
```

Adding rather than discarding duplicates makes the sampled operator an unbiased estimate of its expected value: on a test, the error was 0.005 with summing versus 0.47 with discarding. In the example above, each of the 3 authors is now chosen with probability 1/3. Paths are resampled every training epoch; validation and test use one fixed draw.

After: computation

For a path $v_0, \ldots, v_L$ with step transports $Q_1, \ldots, Q_L$, the transport of node i into the path is $T_i = Q_1 Q_2 \cdots Q_i$. The old code built each T_i separately: L(L+1) transport applications per path. Because each T_i is a scaled rotation ($T_i^\top T_i = s_i^2\, I$), the same result follows from two passes of L steps each:

```
path term at node i:   w_f · ( s_i² x_i − T_i^⊤ z_f ),        z_f = (1/n) Σ_i T_i x_i
forward pass:    Σ_i T_i x_i = x_0 + Q_1( x_1 + Q_2( x_2 + ⋯ + Q_L x_L ) )      (L steps)
backward pass:   T_0^⊤ z = z,   T_i^⊤ z = Q_i^⊤ ( T_{i−1}^⊤ z )                 (L steps)
```

A Paper–Author–Paper path (L = 2) needs 4 transport applications instead of 6, and a path of length 4 needs 8 instead of 20. The output is identical (difference $\leq 2 \cdot 10^{-12}$ in double precision).

Result of 3.1 and 3.2 together

On ACM at d = 32, a training step now uses **1.49 GB and 1.06 s**, down from 2.52 GB and 1.77 s after Section 2, on the same GPU. This figure includes the gates of Section 3.3. The two measurements also differ in the number of sampled paths, so the reduction should not be attributed to one change alone.

3.3 Listen and broadcast gates (proposed, under test)

Before

The diffusion $x \leftarrow x - \eta\, \Delta\hat{}x$ is symmetric: node u pulls node v exactly as hard as v pulls u.

Why we are considering a change

In heterogeneous graphs, influence is plausibly unequal. In ACM, one subject node is linked to as many as 1,195 papers. It should be able to inform its papers without being pulled toward each of them. Under heterophily, a node should also be able to stop listening to dissimilar neighbours. This is a hypothesis, not yet an observed effect.

Proposal

Each node gets a *listen* gate g and a *broadcast* gate b, both between 0 and 1, computed from its input features h_0:

```
g_v = σ(a_gᵀ h₀_v + β_g),     b_v = σ(a_bᵀ h₀_v + β_b)
x ← x − η · G Δ̂B x,           G = diag(g),  B = diag(b)
influence of v on u  =  g_u · b_v          (no longer symmetric)
```

Example: if a subject node learns b ≈ 1 and g ≈ 0.1, it informs its papers fully but is pulled only weakly by them.

Non-symmetric operators usually have complex eigenvalues and can be unstable. This one cannot, because it is similar to a symmetric matrix:

```
with S = (G⁻¹B)^{1/2}:     S (G Δ̂B) S⁻¹  =  (GB)^{1/2} Δ̂(GB)^{1/2}      (symmetric,
positive semidefinite)
⇒ all eigenvalues are real and lie in [0, 2·max g·b) ⊂ [0, 2)  ⇒  the step is stable
for any η < 1
```

On 200 random test graphs, the largest imaginary part of any eigenvalue was $7 \cdot 10^{-17}$. The comparable operator of Cooperative Sheaf Neural Networks reached $2.6 \cdot 10^{4}$ on the same graphs. The gates add 0.6% to the time and 0.5% to the memory of a training step.

Status. We do not yet know whether the gates improve accuracy. The gates are currently part of the model. They will be compared against the symmetric model (all gates equal to 1) over 10 seeds with paired tests, and that comparison will be reported in the paper.

4. Cost in big-O terms

	Old model	New model
Memory for restriction maps	$O(M \cdot d^2)$	$O(M \cdot k \cdot d + \lvert R \rvert \cdot d^2)$
Memory on the path side	$O(F \cdot L \cdot d^2)$	nothing stored (computed on the fly)
Time per diffusion step	at least $O((M + F \cdot L) \cdot d^2)$	$O((M + F \cdot L) \cdot d^2)$
Transport applications per path	$L(L+1)$	$2L$
Number of edges M on ACM	547,872	273,936 (each fact once)

M = number of edges, F = number of path instances, L = path length, |R| = number of relation types, k = 2.

The asymptotic saving is in **memory**: from d^2 numbers per edge to about $k \cdot d$. Time per step keeps the same order, because applying a rotation to a vector still costs d^2. The measured speed-up (up to 4.9×) comes from no longer generating and reading a full

matrix for every edge, and from grouping edges by relation type so the shared rotations are applied in large batches.

5. Next steps

1. Evaluate accuracy on the standard benchmarks (DBLP, ACM, IMDB; 10 seeds) against the old model and the baselines, together with ablations that isolate the effect of the faces and of the intermediate nodes. Two of them test the points above: the new face coupling against the old one (the closure edge as a full edge of the graph operator), and the face-fixed map of the meta-path relation against a freely learned one.
2. Measure how memory and time grow with graph size, path length and number of sampled paths, to confirm the big-O behaviour.
3. Move to larger heterophilic benchmarks (ogbn-mag, mag-year, H-Pokec), which the new memory footprint makes feasible.
4. Rerun link prediction and MUTAG for all methods after the data fixes in Section 3.1, and test whether the listen and broadcast gates help.